

CineMatch — Collaborative Filtering (LLD)

ML Collaborative Filtering — Low Level Design (LLD)

Project: CineMatch / CineWatch AI

Scope: ML Engineer (Collaborative Filtering) deliverables

Files covered: `data_loader.py` , `collaborative_engine.py` , `hybrid_engine.py`

1) Overview

This document describes the low-level design for the **Collaborative Filtering subsystem** and its integration into the **Hybrid Recommendation Engine**.

Goals

- Train an SVD-based collaborative filtering model using MovieLens-style ratings data.
 - Validate model quality using cross-validation (RMSE/MAE).
 - Provide reusable inference methods for predicted rating and normalized score.
 - Integrate collaborative signals into the hybrid recommender (α/β fusion).
 - Enforce “don’t recommend already watched” filtering and sensible cold-start behavior.
-

2) Module: `data_loader.py` — Data Contracts & Preparation

2.1 Responsibility

`DataLoader` prepares and serves data for both training and inference by:

- Loading and cleaning `movies.csv` and `ratings.csv`

- Creating movie **feature bags** (used by content-based engine)
- Building **ID mapping dictionaries** (Movie_ID ↔ index ↔ title)
- Converting ratings to a **Surprise Dataset** for SVD

2.2 Inputs

`ratings.csv` (required schema)

- `userId` (int)
- `movieId` (int)
- `rating` (float)

`movies.csv` (expected columns used)

- `Movie_ID` , `Movie_Title` , `Movie_Genre` , `Movie_Keywords`
- `Movie_Cast` , `Movie_Director` , `Movie_Overview` , `Movie_Tagline` (optional)
- `Movie_Vote` , `Movie_Vote_Count` (optional)
- `Movie_Release_Date` (optional; used to derive year)
- `Movie_Popularity` (used in candidate fallback in `HybridEngine`)

2.3 Key attributes (state)

- `movies_df: pd.DataFrame | None`
- `ratings_df: pd.DataFrame | None`
- `surprise_data: surprise.Dataset | None`

Mappings / indices:

- `movie_id_to_idx: Dict[int, int]`
- `idx_to_movie_id: Dict[int, int]`
- `movie_id_to_title: Dict[int, str]`
- `title_to_movie_id: Dict[str, int]`

Metadata cache:

- `movie_meta: Dict[int, dict]` containing:
 - `movieId`, `title`, `genres`, `director`, `cast` (top 3), `voteAverage`, `voteCount`, `year`, `overview`

2.4 Function-level design

A) `load_movies() -> pd.DataFrame`

Responsibilities

1. Loads `movies.csv`.
2. Cleans text columns by `fillna('')` and `astype(str)`.
3. Converts numeric columns safely:
 - `Movie_Vote` → float (default 0.0)
 - `Movie_Vote_Count` → int (default 0)
4. Extracts year from `Movie_Release_Date` into `Release_Year` (default 2000).
5. Creates the `Feature_Bag`:
 - `Movie_Title + Movie_Genre + Movie_Keywords + Movie_Cast + Movie_Director + Movie_Overview`
 - lowercased and stripped
6. Builds mapping dictionaries.
7. Builds `movie_meta` used by the hybrid recommender to enrich outputs.

Failure modes

- Missing file → `FileNotFoundError`
- Missing columns → may raise a pandas `KeyError` during access (acceptable for hackathon scope; production should validate)

B) `load_ratings(sample_size=None) -> pd.DataFrame`

Responsibilities

1. Loads `ratings.csv`.
2. Validates required columns exist: `userId`, `movieId`, `rating`.
3. Type casts to ensure consistent Surprise behavior:
 - `userId / movieId`: int
 - `rating`: float
4. Optional sampling for faster iteration:
 - `df.sample(n=sample_size, random_state=42)`
5. Builds Surprise dataset:
 - `Reader(rating_scale=(0.5, 5.0))`
 - `Dataset.load_from_df(df[['userId', 'movieId', 'rating']], reader)`

Failure modes

- Missing file → `FileNotFoundError`
 - Schema mismatch → `ValueError`
-

C) `get_user_ratings(user_id) -> pd.DataFrame`

Returns all ratings for a user.

- Requires `ratings_df` to be loaded.
-

D) `get_user_top_movies(user_id, min_rating=4.0, top_k=3) -> List[int]`

Selects **anchor movies** to represent user taste:

- Prefers movies rated $\geq$ `min_rating`
 - Falls back to top-rated movies if none meet the threshold
 - Returns up to `top_k` moviedls
-

3) Module: `collaborative_engine.py` — SVD Collaborative Filtering

3.1 Responsibility

`CollaborativeEngine` owns the collaborative model lifecycle:

- Train Surprise SVD (matrix factorization)
- Cross-validate and report RMSE/MAE
- Predict rating for a given `(userId, movieId)`
- Produce a normalized collaborative score for hybrid blending

3.2 Dependencies

- Requires `data_loader.surprise_data` (built by `DataLoader.load_ratings()`)

Config-driven hyperparameters (`config.py`):

- `SVD_N_FACTORS`, `SVD_N_EPOCHS`, `SVD_LR_ALL`, `SVD_REG_ALL`

3.3 Key attributes

- `data_loader: DataLoader`
- `model: surprise.SVD`
- `trainset: surprise.Trainset | None`
- `is_fitted: bool`
- `eval_metrics: Dict[str, float]`

3.4 Function-level design

A) `__init__(data_loader)`

Initializes the SVD model:

- `SVD(n_factors, n_epochs, lr_all, reg_all, random_state=42)`

B) `fit(max_ratings=250000) -> CollaborativeEngine`

Purpose: Train SVD quickly under hackathon constraints.

Flow

1. If `surprise_data` is missing:

- `data_loader.load_ratings(sample_size=max_ratings)`

2. Build full trainset:

- `surprise_data.build_full_trainset()`

3. Fit model:

- `self.model.fit(trainset)`

4. Set `is_fitted = True`

Design rationale

- Sampling ratings reduces compute while maintaining enough learning signal.
-

C) `evaluate(cv=3) -> Dict[str, float]`

Purpose: Benchmark model performance.

- Uses Surprise `cross_validate` with `measures=['RMSE', 'MAE']`
- Stores mean values in `self.eval_metrics`

Outputs:

- `{ 'rmse': <float>, 'mae': <float> }`
-

D) `predict_rating(user_id, movie_id) -> float`

Predicts expected rating for a user–movie pair using:

- `self.model.predict(uid=user_id, iid=movie_id).est`

Guards:

- Requires `is_fitted == True`
-

E) `get_normalized_score(user_id, movie_id) -> float`

Normalizes predicted rating for hybrid fusion:

1. `raw_pred = predict_rating(...)`
2. Clip to `[1.0, 5.0]`
3. Normalize to `[0.0, 1.0]`:
 - `(clipped - 1.0) / 4.0`

Note (consistency):

- Ratings are loaded with `rating_scale=(0.5, 5.0)`, but normalization clips from 1.0–5.0 for stable blending.
For strict consistency, an alternative is `(x-0.5)/4.5`. Keep current logic if demo UX prefers it.

4) Module: `hybrid_engine.py` — CF Integration into Product Recommendations

4.1 Responsibility

`HybridEngine` converts collaborative predictions into a ranked **Top-N watchlist** by:

- filtering watched titles,
- generating candidate pools,
- scoring with collaborative + content signals,
- blending scores with α/β weights,
- enriching output with metadata needed by UI/API.

4.2 Dependencies

- `DataLoader`:
 - `get_user_ratings`, `get_user_top_movies`, `movies_df`, `movie_meta`, `movie_id_to_idx`
- `CollaborativeEngine`:
 - `is_fitted`, `predict_rating`, `get_normalized_score`
- `ContentEngine` (external):
 - `is_fitted`

- `get_content_recommendations(anchor_id, top_n)`
- `get_content_affinity(anchor_ids, movie_id)`

Config weights (`config.py`):

- `ALPHA_COLLAB` (default 0.6)
- `BETA_CONTENT` (default 0.4)
- `TOP_N_DEFAULT`

4.3 Function-level design

A) `get_user_watched_ids(user_id) -> Set[int]`

Returns set of movie IDs already rated by the user.

- This prevents “recommending what the user already watched”.

B) `get_candidate_movie_ids(user_id, watched_ids, max_candidates=250) -> (List[int], List[int])`

Candidate retrieval strategy combines two sources:

1. Anchor-based content pull

- Get anchor_ids (top 3 favorites ≥ 4.0 ★ when possible)
- For each anchor:
 - `content_engine.get_content_recommendations(anchor_id, top_n=35)`
- Adds a candidate if:
 - not watched
 - exists in `movie_id_to_idx` (valid ID in dataset)

2. Popularity fallback for diversity

- From `movies_df`, take top popular movies:
 - sort by `Movie_Popularity` desc
 - take head(150)

- Add unseen valid items until pool reaches `max_candidates`

Returns:

- `candidates`: list of unique candidate movieids
 - `anchor_ids`: anchors used for taste profile + affinity computation
-

C) `recommend(user_id, top_n=TOP_N_DEFAULT) -> List[dict]`

End-to-end recommendation pipeline:

1. Guardrails:

- content engine must be fitted
- collaborative engine must be fitted

2. Compute:

- `watched_ids`
- `candidates, anchor_ids`

3. Cold-start / empty candidate fallback:

- if `candidates` empty:
 - use top popular movies as candidates
 - set `anchor_ids = []`

4. For each candidate `m_id`:

- Collaborative:
 - `collab_score = collab_engine.get_normalized_score(user_id, m_id)`
 - `pred_rating = collab_engine.predict_rating(user_id, m_id)`
- Content:
 - if anchors exist: `content_score = content_engine.get_content_affinity(anchor_ids, m_id)`
 - else: `content_score = 0.5` (neutral baseline)
- Hybrid fusion:

- `hybrid_score = alpha*collab_score + beta*content_score`
- UI calibration:
 - `matchPercentage = clip(55 + hybrid_score*50, 65, 98)`
 - (This is a display metric for demo UX, not a probability.)
- Metadata enrichment:
 - `meta = data_loader.movie_meta.get(m_id, {})`

5. Sort results by `hybridScore` desc and return top_n.

Per-item output contract

- `movieId`, `title`, `year`, `genres`, `director`, `cast`
 - `predictedRating`, `matchPercentage`
 - `collabScore`, `contentScore`, `hybridScore`
 - `anchorIds`
-

5) End-to-End Sequence (Collaborative Filtering Role View)

5.1 Training & benchmarking

1. `DataLoader.load_movies()`
2. `DataLoader.load_ratings(sample_size=max_ratings)`
3. `CollaborativeEngine.fit(max_ratings=...)`
4. `CollaborativeEngine.evaluate(cv=...)` → RMSE/MAE

5.2 Inference (serving recommendations)

1. `HybridEngine.recommend(user_id)`
2. Candidate generation + watched filtering
3. SVD predicted ratings + normalization
4. Fusion with content affinity

5. Return Top-N ranked list with metadata + explanation hooks (`anchorIds` ,
scores)
-